

Reviewer Pack — Minimal Alexander-Dirac Invariant and Circuit Monotone Width...

Entry 79f985629327 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 23:02:26 UTC

Minimal Alexander-Dirac Invariant and Circuit Monotone Width Correlation

Entry ID: 79f985629327 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 23:02:26 UTC

1. Conjecture

Field A (mathematical branch): Alexander-Dirac Theory **Field B** (complexity object): Boolean Circuits

Statement:

For every d -regular graph G , the minimal Alexander-Dirac invariant ($m_{\text{alex}}(G)$) of its associated 2-dimensional manifold is linearly correlated with its circuit monotone width w_G such that $m_{\text{alex}}(G) = \Theta(w_G)$. Furthermore, for all instances with $d \leq 4$, there exists a constructive mapping from G to a 2-manifold M such that $|w_G - m_{\text{alex}}(M)| \leq 5$.

Rationale (proposer's reasoning):

Alexander-Dirac theory provides an algebraic-topological invariant that measures the complexity of a 2-dimensional manifold. If this invariant can be shown to correlate with circuit monotone width, it might suggest a deeper connection between topology and circuit complexity, potentially revealing new insights into the structure of circuits.

Taxonomy category: Alexander-Dirac (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bb55f743c48929be

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every d -regular graph G with $n \leq 40$, if the correlation coefficient between the Alexander-Dirac invariant ($m_{\text{alex}}(M)$) and the circuit monotone width (w_G) of its associated 2-manifold M is greater than or equal to 0.7 and the absolute difference $|w_G - m_{\text{alex}}(M)|$ is less than or equal to 5, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Alexander-Dirac Theory AND Boolean Circuits - minimal Alexander-Dirac invariant circuit monotone width correlation - d-regular graph 2-dimensional manifold circuit monotone width

Top relevant hits considered: - [<http://arxiv.org/abs/1911.04516v1>] Boolean lattices in finite alternating and symmetric groups - [<http://arxiv.org/abs/1406.3065v2>] Lower Bounds for Tropical Circuits and Dynamic Programs - [<http://arxiv.org/abs/2406.18700v4>] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [<http://arxiv.org/abs/2411.08735v3>] New advances in universal approximation with neural networks of minimal width - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/1903.12062v2>] Lectures on Minimal Surfaces - [<http://arxiv.org/abs/0711.2360v2>] Every longest circuit of a 3-connected, $K_{3,3}$ -minor free graph has a chord - [<http://arxiv.org/abs/cs/0406024v1>] Layout of Graphs with Bounded Tree-Width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A
```

```

def matrix_multiply(A, B):
    m, n, p = len(A), len(B[0]), len(B)
    C = [[0]*n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = Fraction(0)
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        sign = (-1) ** j
        det += sign * A[0][j] * determinant(submatrix)
    return det

def inverse(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    det = determinant(A)
    if det == 0:
        raise ValueError("Singular matrix")
    adjugate = [[0]*n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            sign = (-1) ** (i+j)
            adjugate[j][i] = sign * determinant(submatrix)
    return matrix_multiply(adjugate, Fraction(1, det))

def gaussian_elimination_with_back_substitution(A, b):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot
        b[i] /= pivot
        for j in range(m):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

```

```

        b[j] -= factor * b[i]
    return [b[i] for i in range(m)]

def construct_manifold(G):
    n = len(G)
    M = [[0]*n for _ in range(n)]
    for u in range(n):
        for v in range(u+1, n):
            if G[u][v]:
                M[u][v] = 1
                M[v][u] = 1
    return M

def circuit_monotone_width(G):
    n = len(G)
    max_width = 0
    for i in range(n):
        width = 0
        visited = [False]*n
        stack = [i]
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                width += 1
                for v in range(n):
                    if G[u][v] and not visited[v]:
                        stack.append(v)
        max_width = max(max_width, width)
    return max_width

def alexander_dirac_invariant(M):
    n = len(M)
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if M[i][j]:
                A[i][j] = 1
                A[j][i] = 1
    det_A = determinant(A)
    inv_A = inverse(A)
    B = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if M[i][j]:
                B[i][j] = -inv_A[i][j]
                B[j][i] = inv_A[i][j]
    det_B = determinant(B)
    return abs(det_A + det_B)

def generate_d_regular_graph(n, d):
    G = [[0]*n for _ in range(n)]
    edges = set()
    while len(edges) < n*d//2:
        u = random.randint(0, n-1)
        v = random.randint(0, n-1)

```

```

        if u != v and (u, v) not in edges and (v, u) not in edges:
            G[u][v] = 1
            G[v][u] = 1
            edges.add((u, v))
    return G

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    d = random.randint(2, 4)
    G = generate_d_regular_graph(n, d)
    M = construct_manifold(G)

    w_G = circuit_monotone_width(G)
    m_alex_M = alexander_dirac_invariant(M)

    return {
        "metric_name": "correlation",
        "metric_value": abs(w_G - m_alex_M),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(w_G - m_alex_M) <= 5,
        "counterexample": ""
    }

return run_trial(seed)

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"abs(w_G - m_alex(M)) > 5\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11835
2	propose	ollama_remote	glm4:latest	0	0	12885
3	preregistration	ollama_remote	glm4:latest	0	0	9999
4	novelty	ollama_remote	glm4:latest	0	0	9264
5	novelty	ollama_remote	glm4:latest	0	0	10020
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19458
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10906
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17524
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20776
10	judge	ollama_remote	glm4:latest	0	0	8713

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131379 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/79f985629327.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/79f985629327.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/79f985629327.tar.gz` (if generated)